UNIVERSIDAD AUTÓNOMA “JUAN MISAEL SARACHO”

FACULTAD DE CIENCIAS Y TECNOLOGÍA

DIRECCIÓN DE POSGRADO

CARRERA DE INGENIERÍA INFORMÁTICA

SISTEMA WEB Y MÓVIL PARA PERSONAL VETERINARIO Y PROPIETARIOS DE LA VETERINARIA EIVET EN TARIJA

Trabajo final presentado para la obtención del título de

DIPLOMADO EN DESARROLLO WEB Y APLICACIONES MÓVILES

Postulante: Cristian Arturo Vedia Salazar

Tutor: [NOMBRE DEL TUTOR POR CONFIRMAR]

Tarija – Bolivia

2026

CONTRATAPA

SISTEMA WEB Y MÓVIL PARA PERSONAL VETERINARIO Y PROPIETARIOS DE LA VETERINARIA EIVET EN TARIJA

Postulante: Cristian Arturo Vedia Salazar

Tutor: [NOMBRE DEL TUTOR POR CONFIRMAR]

Autoridades y tribunal: [NÓMINA INSTITUCIONAL POR CONFIRMAR]

Tarija – Bolivia · 2026

HOJA DE APROBACIÓN

Trabajo final presentado a consideración del tribunal evaluador del Diplomado en Desarrollo Web y Aplicaciones Móviles.

_____ Tutor [FIRMA Y NOMBRE]

_____ Tribunal 1 [FIRMA Y NOMBRE]

_____ Tribunal 2 [FIRMA Y NOMBRE]

HOJA DE ADVERTENCIA

[TEXTO DE ADVERTENCIA INSTITUCIONAL POR CONFIRMAR]

ÍNDICE DE CONTENIDO

Capítulo 1. Introducción 1
1.1 Antecedentes 2
1.2 Descripción del problema 3
1.3 Planteamiento del problema 4
1.4 Justificación del problema 4
1.5 Objetivos 6
Capítulo 2. Marco teórico 8
2.1 Fundamentos conceptuales y tecnológicos 8
2.2 Metodología de desarrollo 10
2.3 Análisis de requisitos 12
2.4 Diseño de la solución 20
2.5 Justificación del stack tecnológico 28
2.6 Implementación 29
2.7 Seguridad 30
2.8 Pruebas 31
2.9 Despliegue 33
Capítulo 3. Conclusiones y recomendaciones 36
3.1 Conclusiones 36
3.2 Recomendaciones 37
Bibliografía consultada 38
Anexos 40

ÍNDICE DE TABLAS

Tabla 1. Soluciones informáticas existentes en el dominio 3
Tabla 2. Puntos críticos y datos publicados de otras veterinarias 4
Tabla 3. Correspondencia objetivo, evidencia y conclusión 7
Tabla 4. Periodos de revisión Kanban y evidencias del repositorio 11
Tabla 5. Actores del sistema 12
Tabla 6. Requisitos funcionales, criterios de aceptación y prioridad 14
Tabla 7. Requisitos no funcionales 15
Tabla 8. Capacidades declaradas fuera de alcance 16
Tabla 9. Componentes y responsabilidades 22
Tabla 10. Diccionario de datos de las entidades principales 23
Tabla 11. Operaciones utilizadas por Flutter y Edge Functions 26
Tabla 12. Stack tecnológico y despliegue real 29
Tabla 13. Casos de prueba ejecutados y pendientes 33

ÍNDICE DE FIGURAS

Figura 1. Tablero Kanban de EIVET 11
Figura 2. Casos de uso y permisos por actor 16
Figura 3. Componentes y protocolos de EIVET 21
Figura 4. Entidades principales y referencias del esquema 23
Figura 5. Wireframes iniciales de la aplicación móvil 26

ÍNDICE DE ANEXOS

Anexo A. Manual de usuario 40
Anexo B. Manual de instalación y despliegue 41
Anexo C. Repositorio y tablero 43
Anexo D. URL y paquete instalable 44
Anexo E. Diagramas y trazabilidad 45
Anexo F. Evidencias, casos de prueba y defensa 50

RESUMEN

La Veterinaria EIVET, en Tarija, requiere organizar información de propietarios, mascotas y atención clínica. Este trabajo desarrolla un sistema web para el personal veterinario y una aplicación móvil de consulta y solicitud de citas para propietarios. El proyecto utilizó Kanban para ordenar tareas y Flutter, Supabase y Netlify para la implementación. El repositorio contiene interfaces, modelo de datos, políticas de seguridad y scripts de comprobación. El 06/10/2026 la suite local de Flutter terminó con siete pruebas aprobadas. En la web pública se observó el rechazo de credenciales inválidas y la función crear-propietario respondió 401 sin token. La ruta de salud respondió según lo esperado a GET y HEAD, pero POST devolvió 200 en vez del 405 previsto por el código local. Treinta solicitudes GET medidas el 03/10/2026 respondieron 200; 29 tardaron tres segundos o menos. Esa medición no evalúa las operaciones clínicas. La separación entre dos propietarios, varios flujos completos en producción y el instalable final aún requieren comprobación.

Palabras clave: veterinaria; aplicación móvil; Flutter; Supabase; Kanban.

CAPÍTULO 1. INTRODUCCIÓN

Actualmente, las clínicas veterinarias requieren gestionar una cantidad creciente de información relacionada con propietarios, pacientes animales, consultas médicas, tratamientos, vacunas y programación de atenciones. La administración de estos procesos frecuentemente puede realizarse mediante registros manuales, archivos físicos o herramientas independientes, generando dificultades para mantener la información organizada, acceder rápidamente a los antecedentes clínicos y realizar un seguimiento adecuado de las atenciones realizadas.

El desarrollo de aplicaciones web y móviles ha permitido implementar soluciones orientadas a la digitalización de procesos administrativos y operativos dentro de diferentes organizaciones. Estas tecnologías facilitan el almacenamiento estructurado de información, permiten el acceso desde distintos dispositivos y proporcionan mecanismos para mejorar la disponibilidad, seguridad y trazabilidad de los datos gestionados por los usuarios autorizados.

En el ámbito veterinario existen diversos sistemas informáticos destinados a la administración de clínicas y consultorios, los cuales incorporan funcionalidades como registro de pacientes, gestión de propietarios, historial médico, agenda de citas y seguimiento de tratamientos. Sin embargo, algunas soluciones están diseñadas como plataformas generales o comerciales, por lo que pueden presentar limitaciones relacionadas con costos, adaptación a procesos específicos, personalización o requerimientos particulares de pequeñas organizaciones veterinarias.

Por esta razón, se plantea el desarrollo de una aplicación web y móvil multiplataforma para la gestión veterinaria de la organización beneficiaria EIVET, que permita administrar la información de propietarios y mascotas, registrar consultas, mantener historiales clínicos y facilitar la gestión de citas.

La aplicación estará disponible mediante una interfaz web y una aplicación móvil, de acuerdo con el alcance del proyecto.

1.1 Antecedentes

Los registros clínicos electrónicos veterinarios permiten recuperar y analizar información de la atención. Anholt et al. (2014) documentaron la extracción de registros de consultorios para su uso en vigilancia y estudios basados en evidencia. Ese antecedente respalda la necesidad de estructurar la información clínica, aunque no aporta mediciones del proceso particular de EIVET.

La gestión veterinaria reúne datos de pacientes animales, titulares, consultas, vacunas y agenda. Para revisar soluciones existentes se consultaron las páginas oficiales de VetCloud, DaySmart Vet y VetiCloud el 3 de octubre de 2026. Estas describen historias clínicas y agenda; algunas incluyen facturación, inventario o colaboración entre clínicas. La comparación se limita a las funciones declaradas por cada proveedor, sin afirmar resultados de implantación en EIVET.

EIVET necesita un alcance local más acotado: panel web para administrador y veterinario, consulta móvil de la mascota por su propietario y solicitudes de citas, con datos compartidos en Supabase. La selección tecnológica y las restricciones se detallan en los requisitos y en el Capítulo 2. Las mediciones del proceso manual todavía requieren observación directa en la clínica.

Solución	Alcance documentado	Qué resuelve	Qué no cubre para EIVET	Fuente consultada
VetCloud	Gestión clínica, fichas y agenda.	Centraliza pacientes y recordatorios.	No acredita una adaptación al proceso específico de EIVET; habría que evaluarla.	VetCloud, https://vetcloud.cl/ (03/10/2026).

Solución	Alcance documentado	Qué resuelve	Qué no cubre para EIVET	Fuente consultada
DaySmart Vet	Historia clínica, citas, comunicación, pagos e inventario.	Integra agenda y registros en la nube.	Incluye procesos comerciales fuera del alcance EIVET; no se evaluó su implantación local.	DaySmart Vet, https://www.daysmart.com/vet/ (03/10/2026).
VetiCloud	Historia clínica compartida, visitas y app para dueños.	Conecta clínicas y propietarios.	Su enfoque multicentro supera el alcance de una clínica; no se evaluó su implantación local.	VetiCloud, https://veticloud.com/es (03/10/2026).

Tabla 1. Soluciones informáticas existentes en el dominio. Fuente: elaboración propia.

1.2 Descripción del problema

El diagnóstico inicial del proyecto describe procedimientos manuales para registrar propietarios, mascotas y atenciones en EIVET. La magnitud de los tiempos, errores y duplicaciones aún requiere observación y medición directa en la clínica. La Tabla 2 presenta resultados publicados de otros establecimientos veterinarios como contexto, sin atribuirlos a EIVET.

Debido a que no se cuenta con una plataforma digital que permita organizar la información generada durante la atención veterinaria, el personal debe realizar búsquedas manuales cuando un cliente solicita información sobre una mascota que ya fue atendida anteriormente. Esta situación puede dificultar el acceso oportuno a los antecedentes médicos, afectar el seguimiento de tratamientos y aumentar el tiempo empleado durante el proceso de atención.

N.º	Aspecto comparable	Resultado publicado (no EIVET)
1	Tiempo para recuperar información del paciente	En 7 centros observados de Ghana, 6 tardaron 0–20 min y 1 tardó 21–40 min (Ahmed et al., 2023).
2	Tiempo para registrar información del paciente	Los 7 centros observados emplearon 21–40 min para registrar datos del paciente (Ahmed et al., 2023).
3	Completitud de los registros clínicos	En 36 consultas de una práctica británica se registró el 64,4 % de los problemas tratados y el 58,3 % de las acciones observadas (Jones-Diette et al., 2017). No mide duplicados.

Tabla 2. Puntos críticos y datos publicados de otras veterinarias. Fuente: Ahmed et al. (2023) y Jones-Diette et al. (2017); no son mediciones de EIVET.

1.3 Planteamiento del problema

El proceso de registro y seguimiento de la atención veterinaria en la **Veterinaria EIVET** se realiza actualmente mediante métodos manuales, lo que genera demoras en la consulta de información de clientes y mascotas, dificultades para mantener organizado el historial médico y riesgo de pérdida o duplicidad de registros. No se cuenta con una plataforma digital que permita centralizar y gestionar la información veterinaria, lo que impide acceder de manera rápida y ordenada a los antecedentes de atención de cada mascota.

1.4 Justificación del problema

Factibilidad técnica

El desarrollo de una aplicación web y móvil para la gestión de información veterinaria es técnicamente factible debido a la disponibilidad de tecnologías actuales para la construcción de sistemas multiplataforma, bases de datos y servicios web. Para el desarrollo del proyecto se cuenta con herramientas de programación, frameworks y plataformas de desarrollo que permiten implementar módulos orientados al registro de clientes, mascotas, consultas y seguimiento de atención veterinaria. Además, se dispone del acceso a la

información necesaria proporcionada por la veterinaria para identificar los procesos actuales y definir los requerimientos del sistema. Debido al tiempo establecido para el desarrollo del Trabajo Final de Diplomado, el alcance será delimitado a las funcionalidades principales de gestión y consulta de información veterinaria, evitando incorporar características que excedan la capacidad de implementación del proyecto.

Pertinencia

El desarrollo de esta solución es pertinente debido a que permitirá atender una necesidad existente dentro de la Veterinaria EIVET, relacionada con la organización y administración de la información generada durante la atención de mascotas. La ausencia de una herramienta digital dificulta la consulta de antecedentes, el seguimiento de tratamientos y la gestión ordenada de los datos de clientes y pacientes. Los principales beneficiarios serán el personal encargado de la atención veterinaria, quienes podrán disponer de información organizada, y los propietarios de mascotas, quienes recibirán una atención basada en datos registrados de manera adecuada.

Impacto esperado

La implementación de la aplicación web y móvil permitirá mejorar la organización de los procesos internos de la veterinaria mediante la centralización de la información relacionada con clientes, mascotas y atenciones realizadas. Se espera reducir el tiempo empleado en la búsqueda de registros, facilitar el seguimiento del historial veterinario y disminuir los problemas asociados al manejo manual de información. Además, la solución proporcionará una base tecnológica que podrá contribuir a mejorar la calidad del servicio ofrecido por la veterinaria y facilitar futuras ampliaciones del sistema.

1.5 Objetivos

1.5.1 Objetivo general

Desarrollar una aplicación web y móvil multiplataforma para gestionar propietarios, mascotas, consultas y citas de la Veterinaria EIVET de la ciudad de Tarija, mediante un panel para el personal autorizado y una aplicación de consulta y solicitud de citas para los propietarios.

1.5.2 Objetivos específicos

1. Analizar los procesos actuales de registro y atención veterinaria en la Veterinaria EIVET para identificar los requisitos funcionales y no funcionales necesarios para el desarrollo de la aplicación web y móvil.

2. Diseñar la arquitectura del sistema, el modelo de datos y las interfaces de usuario de la aplicación web y móvil, considerando las necesidades identificadas en la gestión de clientes, mascotas y atención veterinaria.

3. Implementar los módulos principales de la aplicación web y móvil para el registro y consulta de información relacionada con propietarios, mascotas, consultas y tratamientos veterinarios.

4. Aplicar mecanismos de seguridad para la protección de la información almacenada en el sistema, incorporando autenticación de usuarios, control de acceso y validación de datos.

5. Realizar pruebas funcionales de la aplicación web y móvil para verificar el correcto funcionamiento de los módulos implementados y la respuesta del sistema ante diferentes escenarios de uso.

6. Desplegar la aplicación desarrollada y documentar los procedimientos necesarios para su instalación, configuración y utilización por parte del personal de la Veterinaria EIVET

N.º	Objetivo específico	Evidencia en el Capítulo 2	Conclusión en el Capítulo 3
1	Analizar los procesos actuales de registro y atención veterinaria en la Veterinaria EIVET para identificar los requisitos funcionales y no funcionales necesarios para el desarrollo de la aplicación web y móvil.	2.3 Análisis de requisitos	3.1, conclusión 1
2	Diseñar la arquitectura del sistema, el modelo de datos y las interfaces de usuario de la aplicación web y móvil, considerando las necesidades identificadas en la gestión de clientes, mascotas y atención veterinaria.	2.4 Diseño de la solución	3.1, conclusión 2
3	Implementar los módulos principales de la aplicación web y móvil para el registro y consulta de información relacionada con propietarios, mascotas, consultas y tratamientos veterinarios.	2.6 Implementación	3.1, conclusión 3
4	Aplicar mecanismos de seguridad para la protección de la información almacenada en el sistema, incorporando autenticación de usuarios, control de acceso y validación de datos.	2.7 Seguridad	3.1, conclusión 4
5	Realizar pruebas funcionales de la aplicación web y móvil para verificar el correcto funcionamiento de los módulos implementados y la respuesta del sistema ante diferentes escenarios de uso.	2.8 Pruebas	3.1, conclusión 5
6	Desplegar la aplicación desarrollada y documentar los procedimientos necesarios para su instalación, configuración y utilización por parte del personal de la Veterinaria EIVET	2.6 Implementación y README de despliegue; APK aún sin evidencia de distribución.	3.1, conclusión 6

Tabla 3. Correspondencia objetivo – evidencia – conclusión. Fuente: elaboración propia.

CAPÍTULO 2. MARCO TEÓRICO

2.1 Fundamentos conceptuales y tecnológicos

En la comunicación entre componentes se adoptan solicitudes HTTP y datos JSON. Fielding (2000) expone los principios arquitectónicos de los sistemas distribuidos en red; en EIVET, las rutas concretas y sus permisos se contrastan con el código y las políticas del proyecto, no se deducen solo del modelo teórico.

Un expediente veterinario digital relaciona la identidad del propietario con la mascota y sus atenciones, vacunas y tratamientos. La integridad referencial evita que una cita o consulta apunte a una mascota inexistente. En EIVET, el historial visible al propietario depende de la autorización definida en el esquema y de las políticas de acceso, no solo de filtros visuales.

Flutter 3.44.8 y Dart 3.12.2 permiten mantener una base de código para la interfaz Android y el panel web; este proyecto selecciona la vista mediante kIsWeb. La aplicación web se publica en Netlify y el APK se genera por separado. La documentación de Flutter describe el soporte web desde la misma base de código (Flutter, s. f.).

Supabase Auth gestiona identidades y sesiones mediante JWT; PostgREST expone operaciones CRUD sobre PostgreSQL por HTTPS/JSON. Las políticas Row Level Security (RLS) se evalúan en la base de datos para limitar filas por cuenta y rol. Supabase Storage aloja avatares, y las Edge Functions encapsulan operaciones que requieren lógica de servidor, como crear cuentas de propietario (Supabase, s. f.).

Kanban representa tareas en un flujo visible y limita el trabajo en curso. En este proyecto las columnas Hacer, En curso y Hecho permiten priorizar el desarrollo individual; el límite acordado es dos tarjetas simultáneas. Las

revisiones por fecha sirven para documentar avances, sin convertir el flujo en Scrum ni afirmar sprints no realizados (Kanban Guides, 2025).

2.2 Metodología de desarrollo

Marco de trabajo adoptado: Kanban

Se adopta Kanban para organizar y controlar visualmente las actividades del desarrollo de la aplicación web y móvil para la Veterinaria EIVET. El trabajo se divide en tareas pequeñas y verificables, priorizadas en un tablero con límite de trabajo en curso.

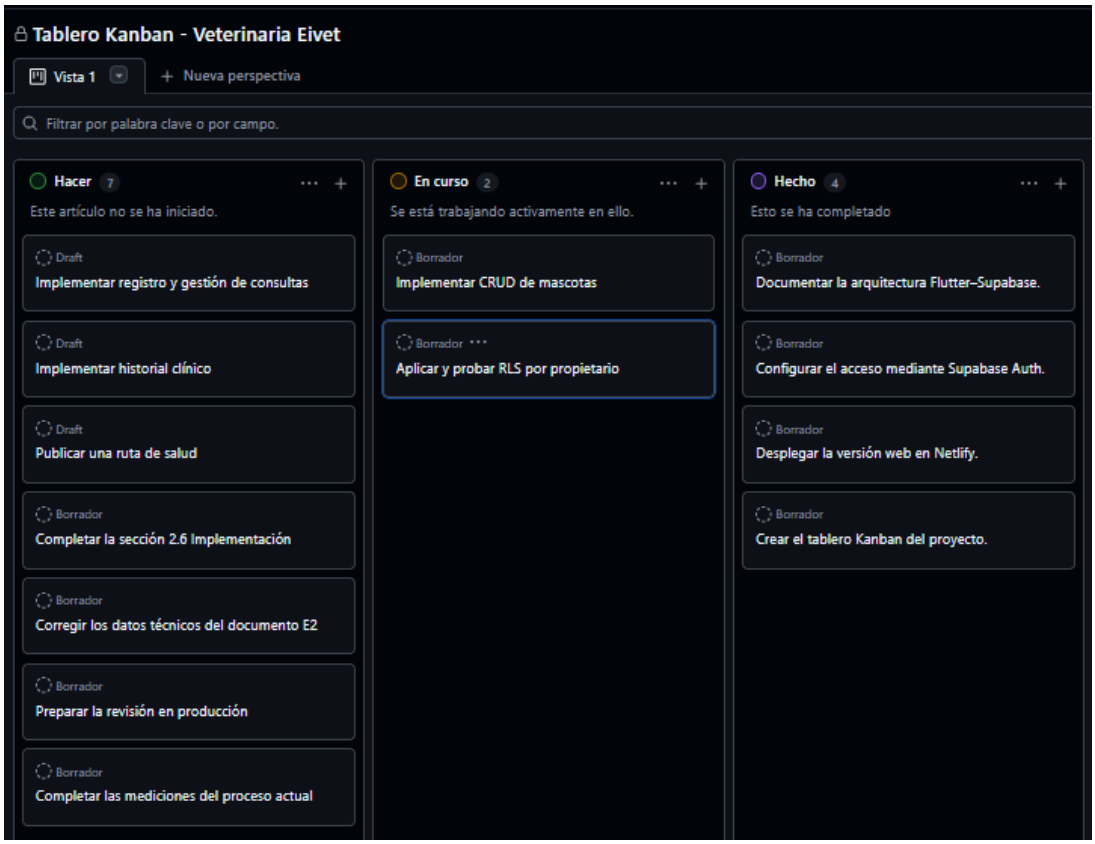
El tablero se revisó al cierre de periodos de trabajo para mover tarjetas según evidencia del repositorio. El flujo fue continuo; no se realizaron ceremonias ni sprints de Scrum. Se limita En curso a dos tarjetas, como muestra la captura conservada de GitHub Projects.

La elección de Kanban resulta adecuada para un proyecto individual, debido a que permite priorizar actividades según su importancia, controlar la cantidad de tareas realizadas simultáneamente y evidenciar el progreso durante las diferentes etapas del desarrollo. Además, facilita la adaptación de las actividades conforme se identifiquen nuevos requerimientos durante el análisis, diseño, implementación y pruebas del sistema.

Iteración de revisión	Periodo documentado	Trabajo observado	Evidencia / estado
1	31 ago – 9 sep 2026	Base móvil y web del proyecto.	Commits a807945 y aababb7; realizado.
2	22 – 24 sep 2026	Estructura y panel web; vinculación de clientes, mascotas y citas.	Commits 8e27770, ed2c194 y 01f9cbd; realizado.
3	25 – 26 sep 2026	Creación de cuentas, correcciones móviles y configuración de Netlify.	Commits b7e8f63 a d8137d4; publicación web observada.
4	3 oct 2026	Ajuste de permisos, ruta de salud y pruebas documentadas.	Commit a1a0acb; pruebas RLS de dos cuentas pendientes.

Tabla 4. Periodos de revisión Kanban y evidencias del repositorio. Fuente: elaboración propia.

Herramienta: GitHub Projects · Tablero: https://github.com/users/arturov7/projects/2/views/1?layout=board · Límite de trabajo en curso: 2 tarjetas. El enlace devuelve 404 sin sesión; debe verificarse el acceso del docente con la cuenta autorizada.



La captura procede de la versión anterior del perfil. La imagen no muestra fecha en pantalla; el estado representado debe cotejarse con GitHub Projects.

Figura 1. Tablero Kanban de EIVET con tareas en Hacer, En curso y Hecho. Fuente: captura del proyecto conservada de la versión anterior del perfil.

2.3 Análisis de requisitos

2.3.1 Actores y perfiles de usuario

ID	Actor	Descripción	Puede realizar	Restricción
A-01	Administrador	Personal autenticado con rol administrador en perfiles.	Crear cuentas de propietario; gestionar datos clínicos y administrativos.	No usa la clave de servicio desde el navegador.
A-02	Veterinario	Personal autenticado con rol veterinario.	Registrar y actualizar mascotas, consultas, tratamientos, vacunas y citas.	La función crear-propietario exige rol administrador.
A-03	Propietario	Cuenta de Auth vinculada a clientes.id.	Consultar sus mascotas y datos autorizados; solicitar y cancelar sus citas.	No registra, edita ni elimina mascotas; no modifica datos clínicos.

Tabla 5. Actores del sistema. Fuente: elaboración propia.

2.3.2 Requisitos funcionales

ID	Requisito funcional	Criterios de aceptación	MoSCoW
RF-01	Como usuario registrado, quiero iniciar sesión para acceder solo a las funciones autorizadas para mi perfil.	CA-01.1: Dado que el usuario tiene credenciales válidas, cuando ingresa sus datos correctamente, entonces el sistema permite el acceso según su perfil. CA-01.2: Dado que el usuario ingresa credenciales incorrectas, cuando intenta iniciar sesión, entonces el sistema muestra un mensaje de error y no permite el acceso.	Must
RF-02	Como personal autorizado, quiero registrar una mascota vinculada a un solo titular; el	CA-02.1: administrador crea la cuenta con datos válidos; personal registra la mascota con cliente_id o propietario_id.	Must

ID	Requisito funcional	Criterios de aceptación	MoSCoW
	administrador puede crear la cuenta de acceso del propietario mediante la función segura.	CA-02.2: se rechaza una mascota sin titular o con dos titulares; el propietario no puede escribirla.	
RF-03	Como personal veterinario**,** quiero registrar consultas veterinarias**,** para mantener un historial de atención de cada mascota**.**	CA-03.1: Dado que existe una mascota registrada, cuando se registra una consulta con sus datos correspondientes, entonces el sistema guarda la atención realizada. CA-03.2: Dado que no existe una mascota asociada, cuando se intenta registrar una consulta, entonces el sistema impide guardar la información.	Must
RF-04	Como personal veterinario, quiero consultar el historial médico de una mascota**,** para revisar antecedentes antes de una nueva atención	CA-04.1: Dado que una mascota posee registros previos, cuando el usuario consulta su historial, entonces el sistema muestra las atenciones registradas. CA-04.2: Dado que una mascota no tiene historial registrado, cuando se realiza la consulta, entonces el sistema informa que no existen registros disponibles.	Must
RF-05	Como personal veterinario, quiero registrar tratamientos y vacunas aplicadas**,** para llevar seguimiento de los cuidados realizados a cada mascota**.**	CA-05.1: Dado que existe una atención registrada, cuando se agrega un tratamiento o vacuna, entonces el sistema almacena la información asociada a la mascota. CA-05.2: Dado que faltan datos requeridos del tratamiento, cuando se intenta guardar, entonces el sistema muestra una validación indicando la información pendiente.	Should
RF-06	Como propietario de mascota**,** quiero consultar la información de mis mascotas**,** para conocer sus datos y antecedentes disponibles**.**	CA-06.1: Dado que el propietario está registrado, cuando accede al sistema, entonces puede visualizar únicamente la información asociada a sus mascotas. CA-06.2: Dado que intenta consultar información de otra mascota, cuando realiza la búsqueda, entonces el sistema restringe el acceso.	Should
RF-07	Como personal veterinario**,** quiero actualizar información de clientes, mascotas y atenciones, para mantener los registros actualizados.	CA-07.1: Dado que existe un registro almacenado, cuando el usuario modifica información válida, entonces el sistema actualiza los datos correctamente.	Could

ID	Requisito funcional	Criterios de aceptación	MoSCoW
		CA-07.2: Dado que se ingresan datos inválidos, cuando se intenta actualizar un registro, entonces el sistema rechaza la modificación e indica el error.	
RF-08	Como propietario de mascota**,** quiero solicitar una cita veterinaria**,** para facilitar la coordinación de una atención**.**	CA-08.1: Dado que el propietario está autenticado, cuando completa la fecha, motivo y datos requeridos para solicitar una cita, entonces el sistema registra la solicitud con estado pendiente. CA-08.2: Dado que faltan datos obligatorios o el usuario no está autenticado, cuando intenta enviar la solicitud, entonces el sistema la rechaza e indica qué dato debe corregirse.	Could

Tabla 6. Requisitos funcionales, criterios de aceptación y prioridad. Fuente: elaboración propia.

Total de requisitos: 8 · Must have: 4 · Porcentaje Must have: 50 % (debe ser ≤ 60 %).

2.3.3 Requisitos no funcionales

ID	Tipo	Enunciado	Métrica de verificación
RNF-01	Rendimiento	El sistema debe responder a las operaciones principales de consulta, registro y actualización de información en tiempos adecuados para garantizar una experiencia fluida al usuario.	El 90 % de las solicitudes realizadas durante las pruebas funcionales debe obtener respuesta en un tiempo menor o igual a 3 segundos bajo condiciones normales de uso.
RNF-02	Seguridad	El sistema debe proteger la información gestionada mediante autenticación de usuarios, control de acceso según rol y protección de datos sensibles.	En las pruebas, 100 % de los intentos de leer o modificar datos de otro propietario deben ser rechazados por RLS; las contraseñas se

ID	Tipo	Enunciado	Métrica de verificación
			gestionan exclusivamente en Supabase Auth y nunca se almacenan en tablas de la aplicación.
RNF-03	Usabilidad	El sistema debe proporcionar una interfaz clara e intuitiva que permita a los usuarios realizar las operaciones principales sin requerir capacitación extensa.	Al menos el 80 % de los usuarios de prueba debe completar las tareas principales definidas sin asistencia externa y con una navegación comprensible.
RNF-04	Compatibilidad	El sistema debe funcionar correctamente en los entornos tecnológicos definidos para su implementación, considerando navegadores o dispositivos objetivo.	La aplicación debe funcionar sin errores críticos en Android 8.0 (API 26) o superior y en Google Chrome 120 o superior; registrar el resultado en una prueba por plataforma.
RNF-05	Disponibilidad	El sistema debe mantenerse accesible durante el periodo de operación establecido para permitir la consulta y gestión de información por parte de los usuarios autorizados.	Objetivo de disponibilidad ≥95 % durante el periodo de evaluación. El plan gratuito de Supabase puede pausar proyectos inactivos; durante la evaluación se verificará el estado antes de las pruebas y se documentará cualquier pausa. No se afirma disponibilidad continua fuera del periodo observado.

Tabla 7. Requisitos no funcionales. Fuente: elaboración propia.

Alcance y exclusiones

Capacidad excluida	MoSCoW	Justificación de la exclusión
Módulo de venta de productos veterinarios, incluyendo catálogo de productos, carrito de compra, facturación y gestión de pagos	Won't	La incorporación de ventas requeriría implementar procesos comerciales adicionales como control de inventario, precios, facturación y medios de pago, los cuales no forman parte del problema identificado.

Capacidad excluida	MoSCoW	Justificación de la exclusión
Gestión avanzada de inventario de medicamentos, alimentos y accesorios veterinarios	Won't	Esta capacidad podría incorporarse como una ampliación futura; sin embargo, requiere definir procesos específicos de abastecimiento, entradas y salidas de productos, alertas de stock y controles administrativos que exceden el alcance actual.
Integración con plataformas externas de pago o servicios comerciales.	Won't	Se considera una mejora futura debido a que actualmente el sistema no contempla operaciones comerciales. Su implementación dependería de la incorporación previa de módulos de venta y acuerdos con proveedores externos.

Tabla 8. Capacidades declaradas fuera de alcance. Fuente: elaboración propia.

2.3.4 Casos de uso

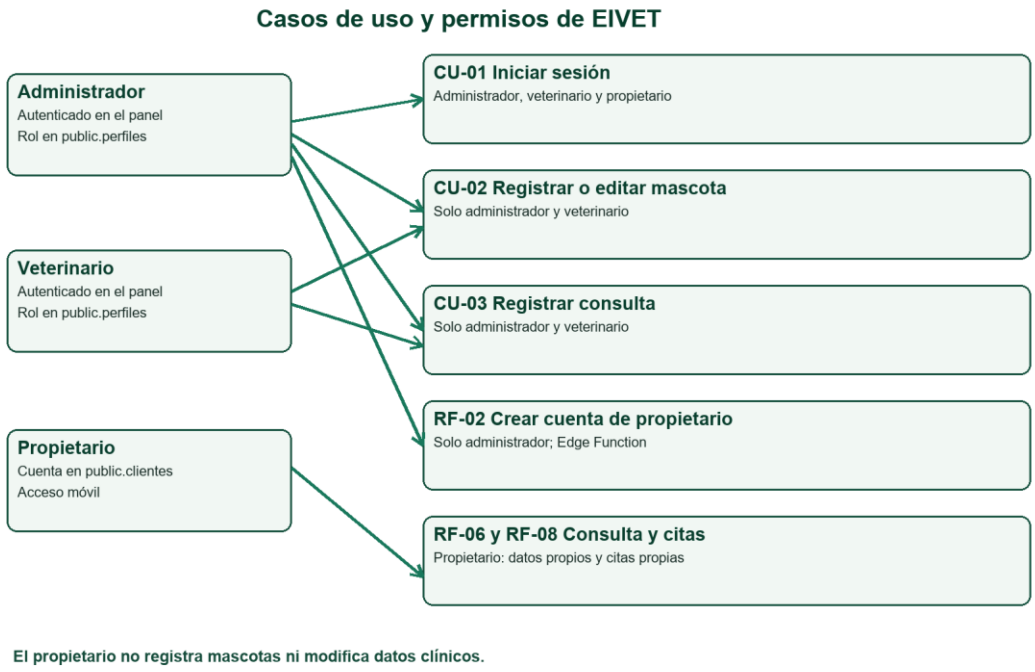


Figura 2. Casos de uso y permisos por actor. Fuente: elaboración propia según RF-01 a RF-08 y función crear-propietario.

CU-01 Iniciar sesión

Actor principal:

Personal veterinario / Propietario de mascota

Precondición:

El usuario debe encontrarse registrado previamente en el sistema y contar con credenciales válidas.

Flujo principal:

1. El usuario accede a la pantalla de inicio de sesión.
2. El sistema solicita correo electrónico y contraseña.
3. El usuario introduce sus credenciales.
4. El sistema valida la información ingresada.
5. El sistema identifica el perfil del usuario.
6. El sistema permite el acceso según los permisos asignados.

Flujos alternativos:

A1. Si las credenciales ingresadas son incorrectas, el sistema muestra un mensaje indicando que los datos no son válidos y mantiene bloqueado el acceso.

A2. Si el usuario no cuenta con permisos suficientes para acceder a una función, el sistema restringe la operación solicitada.

CU-02 Registrar mascota

Actor principal: administrador o veterinario autenticado. Solo el administrador puede crear una cuenta de acceso nueva.

Precondición: existe una cuenta en clientes o una ficha manual en propietarios; si se necesita cuenta nueva, el administrador la crea primero.

El personal abre Mascotas en el panel web y elige registrar.

1. El sistema muestra el formulario de mascota y solicita seleccionar exactamente un titular.

2. El personal completa nombre y demás datos disponibles.
3. El sistema valida los campos obligatorios y el vínculo con el titular.
4. El personal confirma el registro de la mascota.
5. PostgREST almacena la mascota si el rol y las políticas RLS lo permiten.
6. Postcondición: la mascota queda vinculada al titular seleccionado.

Flujos alternativos:

A1. Si la cuenta no existe, el administrador la crea mediante crear-propietario o selecciona una ficha manual autorizada.

A2. Si se seleccionan dos titulares o ninguno, el registro se rechaza. Si el usuario es propietario, la escritura debe ser denegada por RLS.

CU-03 Registrar consulta veterinaria

Actor principal:
Personal veterinario

Precondición:
La mascota debe encontrarse registrada previamente.

Flujo principal:

1. El veterinario selecciona una mascota registrada.
2. El sistema muestra la información existente del paciente.
3. El veterinario registra los datos de la consulta.
4. El sistema valida la información ingresada.
5. El sistema guarda la atención realizada.
6. El historial clínico de la mascota se actualiza.

Flujos alternativos:

A1. Si no existe una mascota asociada, el sistema impide registrar la consulta.

A2. Si faltan datos obligatorios de la atención, el sistema solicita completar la información antes de guardar.

Postcondición:
La consulta queda almacenada dentro del historial clínico de la mascota.

2.4 Diseño de la solución

2.4.1 Arquitectura del sistema

Arquitectura cliente–servicio administrado con Flutter, Supabase y Netlify.

El navegador recibe Flutter Web desde Netlify por HTTPS. El teléfono ejecuta Flutter Android. Ambos clientes consumen los servicios de Supabase por HTTPS/JSON; no existe un servidor de aplicación propio.

Supabase Auth emite sesiones; PostgREST expone tablas; Edge Functions ejecuta crear-propietario y salud; PostgreSQL aplica relaciones, restricciones y las políticas RLS instaladas.

Flutter presenta las interfaces para el personal veterinario y el propietario, con validaciones iniciales y consumo de servicios HTTPS de Supabase (RF-01 al RF-08).

1. PostgREST conecta la aplicación con PostgreSQL. El repositorio contiene políticas RLS para autorización por fila; su estado remoto debe verificarse con pruebas de dos cuentas antes de dar por satisfecho RNF-02.
2. PostgreSQL almacena clientes, perfiles, propietarios, mascotas, consultas, tratamientos, vacunas, historiales y citas mediante relaciones. Las reglas de los scripts deben aplicarse en el proyecto remoto para que limiten los registros visibles.
3. Flutter se comunica directamente con Supabase Auth y PostgREST a través de HTTPS con JSON. La aplicación usa únicamente la publishable/anon key; la clave service_role permanece exclusivamente en entornos de servidor y nunca se incorpora al APK ni al repositorio.
4. La figura etiqueta los intercambios HTTPS/JSON, JWT y SQL interno. La clave de servicio permanece en la Edge Function; la aplicación usa la publishable key.

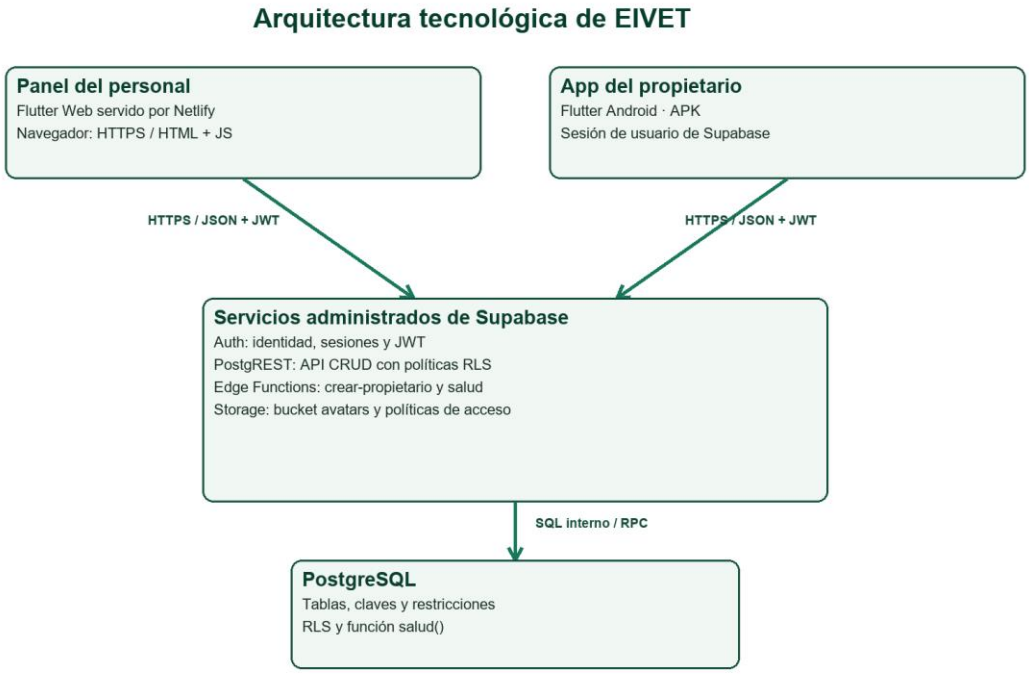


Figura 3. Componentes y protocolos de EIVET. Fuente: elaboración propia a partir de lib/main.dart, supabase/config.toml y netlify.toml.

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
Panel del personal	Interfaz administrativa.	Flutter Web 3.44.8; Netlify	Navegador; Supabase	HTTPS/HTML/JS; HTTPS/JSON + JWT
App del propietario	Consulta y citas.	Flutter Android 3.44.8	Supabase	HTTPS/JSON + JWT
Identidad	Sesiones y JWT.	Supabase Auth	Clientes; auth.users	HTTPS/JSON; JWT
API de datos	CRUD de tablas expuestas.	Supabase PostgREST	Flutter; PostgreSQL	HTTPS/JSON; SQL interno

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
Funciones	Alta segura de cuentas y salud.	Supabase Edge Functions	Auth; RPC salud	HTTPS/JSON; SQL interno
Persistencia	Datos y restricciones por fila.	PostgreSQL + RLS	PostgREST; funciones	SQL interno
Archivos	Avatares y políticas del bucket.	Supabase Storage	App Flutter	HTTPS/API Storage

Tabla 9. Componentes y responsabilidades. Fuente: elaboración propia.

2.4.2 Modelo de datos

El esquema combina cuentas móviles en clientes (id = auth.users.id) y fichas manuales en propietarios. Cada mascota referencia un solo titular mediante cliente_id o propietario_id. Citas conserva el vínculo con mascota y titular; consultas, historiales_clinicos, vacunas y tratamientos referencian mascota. Las vacunas no tienen consulta_id; tratamientos sí admite una consulta_id opcional. Perfiles almacena los roles del personal; nunca guarda contraseñas. El diagrama muestra las entidades centrales y deja servicios y notificaciones como tablas complementarias.

Las políticas previstas en los scripts 12 y 13 limitan escritura de mascotas al personal y lectura del propietario a cliente_id = auth.uid(), además de validar la cita contra la mascota. Los scripts están en el repositorio; la ejecución remota de estas últimas migraciones y la prueba de dos cuentas aún no están documentadas como aprobadas.

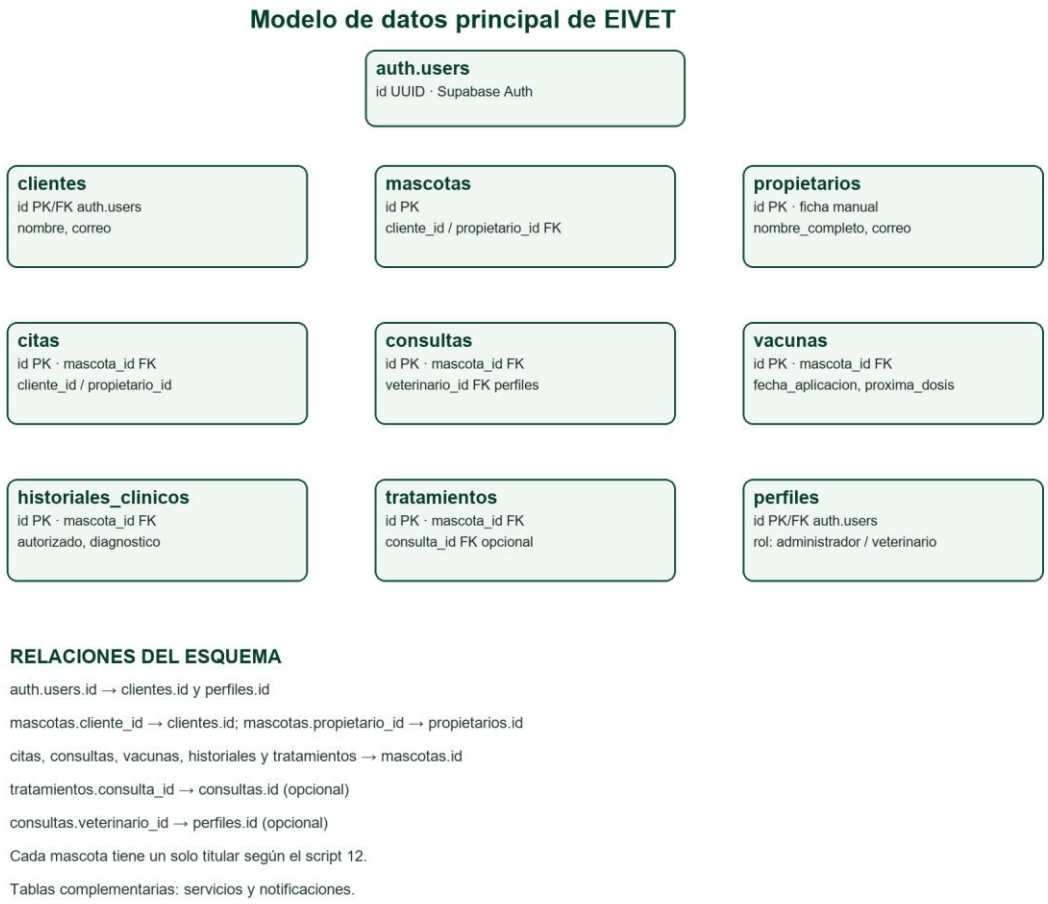


Figura 4. Entidades principales y referencias del esquema de Supabase. Fuente: elaboración propia a partir de los scripts SQL del proyecto.

Tabla 10. Diccionario de datos de las entidades principales. Fuente: scripts SQL del proyecto.

Entidad	Campo	Tipo	Obligatorio	Clave / referencia	Descripción
clientes	id	UUID	Sí	PK; FK auth.users.id	Cuenta móvil autenticada.
clientes	nombre, correo	TEXT	Nombre sí; correo según script 10	—	Datos del propietario con acceso.
perfiles	id, rol	UUID, TEXT	Sí	PK/FK auth.users; CHECK rol	Personal administrador o veterinario.

Entidad	Campo	Tipo	Obligatorio	Clave / referencia	Descripción
propietarios	id, nombre_completo	UUID, TEXT	Sí	PK	Ficha manual, sin cuenta de Auth.
mascotas	id, nombre	UUID, TEXT	Sí	PK	Paciente animal.
mascotas	cliente_id / propietario_id	UUID	Uno de dos	FK clientes / propietarios	Titular único con script 12.
mascotas	especie, raza, sexo, fecha_nacimiento, peso	TEXT, DATE, NUMERIC	No	—	Datos descriptivos.
citas	id, mascota_id	UUID	Sí	PK; FK mascotas	Solicitud o atención agendada.
citas	cliente_id / propietario_id	UUID	Según titular	FK clientes / propietarios	Titular relacionado.
citas	fecha_hora, motivo, estado	TIMESTAMPTZ, TEXT	Sí	CHECK estado	Agenda y estado.
consultas	id, mascota_id, veterinario_id	UUID	Mascota sí	PK; FK mascotas / perfiles	Atención del panel.
consultas	fecha, motivo, diagnostico	DATE, TEXT	Fecha y motivo sí	—	Datos de consulta.
historiales_clinicos	id, mascota_id, autorizado	UUID, BOOLEAN	Sí	PK; FK mascotas	Historial visible según autorización.
vacunas	id, mascota_id, nombre	UUID, TEXT	Sí	PK; FK mascotas	Vacuna registrada.
vacunas	fecha_aplicacion, proxima_dosis	DATE	No	—	Fechas de seguimiento.
tratamientos	id, mascota_id, consulta_id	UUID	Mascota sí	PK; FK mascota; consulta opcional	Tratamiento actual o previo.
servicios	id, nombre, activo	UUID, TEXT, BOOLEAN	Sí	PK	Catálogo de servicios.
notificaciones	id, cliente_id, leida	UUID, BOOLEAN	Sí	PK; FK clientes	Mensajes del cliente.

2.4.3 Diseño de la interfaz

Figura 5. Wireframes de las pantallas principales

Aplicación móvil Veterinaria **EIVET**

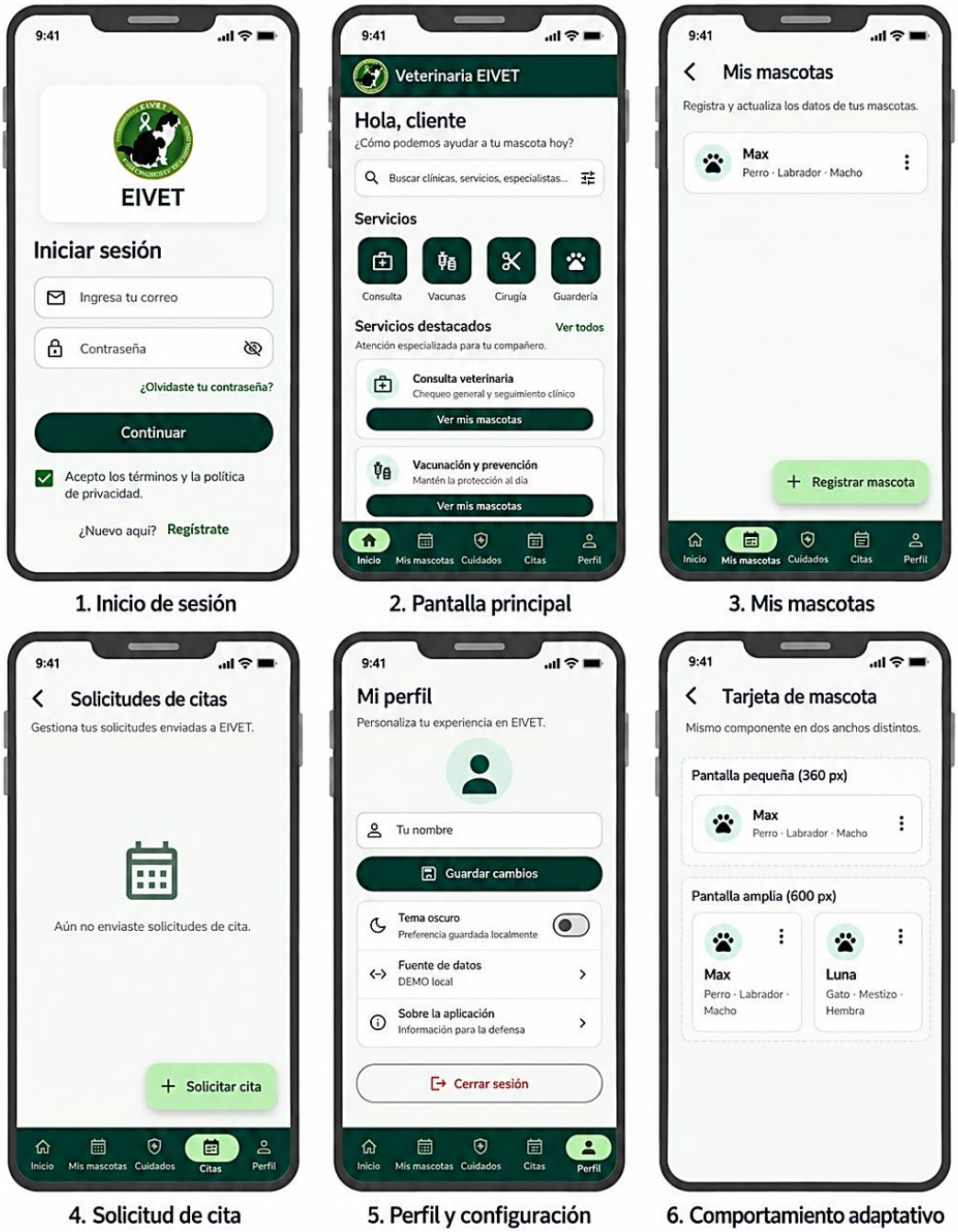


Figura 5. Wireframes iniciales de la aplicación móvil. La opción de registrar mascotas para el propietario es un diseño previo y no corresponde a los permisos vigentes. Fuente: elaboración propia.

2.4.4 Diseño de la interfaz de programación de aplicaciones

Tabla 11. Operaciones utilizadas por Flutter y Edge Functions. Fuente: código del proyecto.

Método	Ruta	Auth	Rol	Parámetros	2xx esperado	Error / límite
POST	/auth/v1/token?grant_type=password	No	Cuenta existente	correo, contraseña	Sesión JWT	Credenciales inválidas
POST	/auth/v1/signup	No	Propietario	correo, contraseña	Usuario creado	Validación o correo duplicado
POST	/functions/v1/crear-propietario	JWT	Administrador	nombre, correo, contraseña, contacto	201 + id	400, 401, 403, 409, 500
GET	/rest/v1/clientes y /perfiles	JWT	Personal	select y filtros	Lista visible	RLS / permisos
GET/POST/PATCH/DELETE	/rest/v1/propietarios	JWT	Personal	id y JSON	Datos / 2xx	RLS / validación
GET	/rest/v1/mascotas	JWT	Personal; propietario propio	cliente_id, id, select	Filas visibles	RLS puede devolver 0 filas
POST/PATCH/DELETE	/rest/v1/mascotas	JWT	Administrador o veterinario	titular único; id; JSON	2xx si autorizado	RLS o titular inválido
GET/POST/PATCH/DELETE	/rest/v1/consultas	JWT	Personal	mascota_id, id, JSON	Datos / 2xx	RLS / validación
GET/POST/PATCH/DELETE	/rest/v1/tratamientos y /vacunas	JWT	Personal; lectura autorizada móvil	mascota_id, id, JSON	Datos / 2xx	RLS / validación
GET/POST/PATCH	/rest/v1/citas	JWT	Personal; propietario propio	mascota_id, cliente_id, id, JSON	Datos / 2xx	RLS / mascota ajena

Método	Ruta	Auth	Rol	Parámetros	2xx esperado	Error / límite
GET	/rest/v1/historiales_clinicos	JWT	Personal; propietario autorizado	mascota_id	Historial visible	RLS / autorizado=false
POST/DELETE	/storage/v1/object/avatars	JWT	Dueño del avatar	ruta y archivo	2xx	Política Storage
GET/HEAD	/functions/v1/salud	No	Público	sin datos clínicos	200 online	503 offline; POST 200 observado: fallo

2.5 Justificación del stack tecnológico

Las versiones locales de Flutter y Dart provienen del archivo de caché del SDK; Netlify fija Flutter 3.44.8. La versión de los servicios administrados no se inventa. URL del repositorio: https://github.com/arturov7/Veterinaria-Eivet; URL del tablero: https://github.com/users/arturov7/projects/2/views/1?layout=board; ruta de salud: https://xbvjhrfgznljtwxghvka.supabase.co/functions/v1/salud.

Componente	Versión	Función en el sistema	Criterio de selección
Flutter / Dart	Flutter 3.44.8 / Dart 3.12.2 (caché local y netlify.toml)	Interfaces Android y web.	Una base de código para ambas vistas; RF-01 a RF-08.
supabase_flutter	^2.9.1 en pubspec.yaml; 2.17.2 resuelta en pubspec.lock	Auth, datos y Storage desde Flutter.	Integra servicios gestionados; RF-01 a RF-08.
Supabase Auth	Servicio administrado, sin versión fijada	Identidad y sesiones JWT.	Control de acceso; RNF-02.
PostgreSQL / PostgREST	Versiones remotas no confirmadas	Persistencia y API REST con RLS.	Relaciones y autorización; RNF-02.
Netlify	Servicio de despliegue; Flutter 3.44.8 fijado	Web pública: https://eivetveterinaria.netlify.app/	Publicación desde GitHub; RNF-05.
Supabase Edge Functions	Runtime remoto no confirmado	crear-propietario y salud.	Lógica segura de servidor; RF-02 y monitoreo.
Git / GitHub Projects	Versiones cliente no necesarias para el usuario	Historial y tablero Kanban.	Trazabilidad de iteraciones.

Tabla 12. Stack tecnológico y despliegue real. Fuente: pubspec.yaml, netlify.toml y servicios publicados.

2.6 Implementación

La solución se implementó en Flutter con una entrada común. En la versión web se muestra el panel para el personal y en Android la aplicación del propietario; la selección ocurre en lib/main.dart mediante kIsWeb. La configuración de Supabase se recibe en tiempo de compilación con APP_MODE, SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY. Los repositorios separan las pantallas de las consultas a Supabase y permiten un modo demo para desarrollo. El modo demo no constituye evidencia de funcionamiento de producción.

La aplicación móvil incluye inicio de sesión, consulta de mascotas asociadas a la cuenta, solicitudes de citas y visualización de cuidados. El panel web reúne la gestión de propietarios, mascotas, citas, consultas, vacunas y tratamientos. Supabase Auth gestiona las identidades; las tablas de PostgreSQL almacenan los registros clínicos. La función crear-propietario reserva la creación de cuentas desde el panel a un administrador autenticado y la función salud consulta una RPC de solo lectura. La publicación web se realiza mediante Netlify desde el repositorio GitHub.

Una dificultad real apareció en el formulario móvil de mascotas: valores históricos de sexo con distinta capitalización causaban una aserción del desplegable de Flutter. Se resolvió normalizando el valor antes de asignarlo al control y dejando dos opciones únicas, Macho y Hembra. El cambio consta en el commit 7dd5379. Esta corrección de interfaz no demuestra por sí misma que las restricciones de acceso a mascotas estén activas en el proyecto remoto.

2.7 Seguridad

2.7.1 Autenticación y gestión de sesiones

Integré Supabase Auth para iniciar y cerrar sesión, registrar cuentas móviles y recuperar contraseñas. El cliente observa los cambios de sesión y asocia las preferencias locales con el identificador del usuario. El 06/10/2026 el panel web rechazó credenciales ficticias inválidas y permaneció en la pantalla de acceso; el camino feliz y la caducidad o renovación de una sesión en producción siguen sin prueba documentada.

2.7.2 Autorización por rol

El esquema define los roles administrador y veterinario en public.perfiles. La función crear-propietario valida el JWT y consulta el rol administrador antes de crear una cuenta. La política prevista para mascotas permite al propietario leer las vinculadas con su cuenta y reserva el alta, edición y eliminación al personal. La política prevista para citas también comprueba el vínculo con la mascota. Los scripts 12_MASCOTAS_TITULAR_SOLO_PERSONAL.sql y 13_CITAS_RLS_PROPIETARIOS.sql describen estas reglas, pero su aplicación efectiva y el aislamiento entre dos cuentas siguen pendientes de comprobación en Supabase. Por ello no se afirma todavía que la protección RLS esté aprobada en producción.

2.7.3 Validación de entradas y vulnerabilidades habituales

Los formularios móviles comprueban campos obligatorios antes de guardar. La función crear-propietario rechaza métodos distintos de POST, JSON inválido, correo con formato incorrecto y contraseñas de menos de ocho caracteres; también exige una sesión válida y rol administrador. Comprobé el 06/10/2026 que una petición sin token respondió HTTP 401; no se inspeccionaron Auth ni las tablas para confirmar efectos sobre datos. Las políticas RLS deben impedir el acceso directo no autorizado por PostgREST,

incluso si alguien evita la interfaz. Esa defensa sigue sin prueba con dos cuentas. La ruta salud solo devuelve online u offline y no lee datos clínicos.

2.7.4 Cifrado de credenciales y comunicaciones

Las contraseñas se entregan a Supabase Auth y no se guardan en las tablas de la aplicación. Las URL públicas del panel y de Supabase emplean HTTPS; comprobé su acceso por esa vía. Este trabajo no evalúa los mecanismos internos de cifrado del proveedor ni inspecciona todo el tráfico móvil.

2.7.5 Gestión de variables sensibles

El archivo .env local está excluido del repositorio; Netlify recibe SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY como variables de compilación. La publishable key es visible para el cliente y no se trata como secreto. La clave de servicio solo se obtiene en la Edge Function desde el entorno de Supabase y no debe incluirse en Flutter, el repositorio ni las capturas. No se reproducen valores de claves en este documento.

2.8 Pruebas

El 06/10/2026 ejecuté flutter test: siete pruebas locales aprobaron y el proceso terminó con código de salida 0. Estas pruebas cubren lógica demo y conversión de datos, sin acreditar RLS ni operaciones clínicas de producción. Repetí scripts/verify_salud.ps1 contra la URL pública: GET y HEAD cumplieron, mientras POST respondió 200 en lugar de 405. Además, el panel rechazó credenciales ficticias inválidas y crear-propietario respondió 401 sin token. El reporte local, las salidas de salud y las evidencias de esos dos caminos de error se conservan en docs/evidencias; la matriz completa está en docs/PRUEBAS_Y_EVIDENCIAS.md.

ID	Escenario evaluado	Resultado esperado	Resultado observado	Estado
CP-01	Apertura del panel web	Página pública disponible	Página EIVET observada en Netlify; flujos internos sin probar	Aprobado solo para disponibilidad
CP-02	Salud GET	HTTP 200; status online	HTTP 200; status online	Aprobado
CP-03	Salud HEAD	HTTP 200; sin cuerpo	HTTP 200; sin cuerpo	Aprobado
CP-04	Salud POST	HTTP 405	HTTP 200; status online	Fallido
CP-05	Aislamiento entre dos propietarios	Sin lectura ni edición cruzada	No ejecutado en producción	Pendiente
CP-06	Permisos de mascotas por rol	Personal escribe; propietario solo lee	No ejecutado de extremo a extremo	Pendiente
CP-07	Suite Flutter local	Pruebas sin fallos	7 aprobadas; salida 0 (06/10/2026)	Aprobado local
CP-08	crear-propietario sin token	HTTP 401	HTTP 401 (06/10/2026); efectos en BD no inspeccionados	Aprobado solo para HTTP
CP-09	crear-propietario con veterinario	HTTP 403; sin alta	No ejecutado	Pendiente
CP-10	crear-propietario con administrador	HTTP 201; alta única	No ejecutado	Pendiente
CP-11	RF-01: credenciales inválidas	Rechazo sin acceso	Pantalla de acceso y error visible (06/10/2026)	Aprobado: error
CP-12	RF-02: propietario y mascota	Alta autorizada; denegación al propietario	No ejecutado integralmente	Pendiente

ID	Escenario evaluado	Resultado esperado	Resultado observado	Estado
CP-13	RF-03: consulta	Registro válido; error con mascota inválida	No ejecutado en producción	Pendiente
CP-14	RF-04: historial	Consultas previas; estado vacío	No ejecutado en producción	Pendiente

Tabla 13. Casos de prueba ejecutados y pendientes al 06/10/2026. Fuente: elaboración propia.

Se conserva una captura fechada del rechazo de credenciales inválidas en la web pública. Siguen pendientes capturas fechadas de los flujos clínicos y de los permisos entre cuentas ficticias, así como la comprobación de efectos sobre Auth y la base de datos. La respuesta online de salud acredita la consulta de comprobación a PostgreSQL, pero no el funcionamiento de todos los módulos.

Limitaciones al 06/10/2026: faltan mediciones del proceso actual de EIVET, acceso externo al tablero privado, verificación remota de RLS con dos propietarios ficticios, caminos felices y de error completos de RF-01 a RF-04, capturas fechadas de los flujos clínicos, prueba integral del CRUD y un APK final instalado. La medición de la ruta de salud figura en 2.9.1; sus resultados no prueban el rendimiento del CRUD.

2.9 Despliegue

La versión web se publica desde la rama principal del repositorio en Netlify. El archivo netlify.toml fija Flutter 3.44.8, ejecuta scripts/build_netlify.sh y publica build/web. El proceso obtiene dependencias, compila Flutter Web y entrega archivos estáticos por HTTPS. Las variables SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY se suministran en Netlify; ninguna clave de servicio debe incluirse en el navegador o APK.

Para reconstruir localmente se clona el repositorio, se instala Flutter 3.44.8, se ejecuta flutter pub get y se configura .env con los valores del proyecto Supabase. El panel se inicia con flutter run -d chrome --dart-define-from-file=.env. La aplicación Android se inicia seleccionando el identificador exacto de flutter devices; para generar un APK arm64 se usa flutter build apk --release --target-platform android-arm64 --dart-define-from-file=.env. El paquete resultante se encuentra en build/app/outputs/flutter-apk/app-release.apk. Estos pasos son instrucciones reproducibles; en esta revisión no se ejecutó una compilación nueva.

La base de datos y los permisos se reconstruyen aplicando los scripts SQL del directorio supabase en el orden documentado en docs/SUPABASE_EIVET.md. La ruta de salud requiere la RPC de supabase/salud.sql y la Edge Function supabase/functions/salud; la función crear-propietario utiliza variables exclusivas del servidor. Los despliegues de funciones y migraciones requieren credenciales de administrador y deben verificarse en el proyecto remoto.

La comprobación pública del 03/10/2026 confirmó acceso al sitio web y 30 respuestas HTTP 200 de GET /functions/v1/salud. El 06/10/2026 repetí la prueba de métodos: GET y HEAD cumplieron, pero POST aún devolvió 200 en lugar del 405 esperado por el código local. No se había sincronizado la Edge Function publicada. Tampoco se confirmó en producción la aplicación de las últimas políticas RLS, el CRUD completo, un APK final ni las pruebas con dos propietarios. El enlace del tablero requiere permiso para terceros.

El procedimiento de verificación posterior al despliegue debe abrir el sitio, iniciar sesión con cuentas ficticias de cada rol, recorrer un alta y consulta autorizadas, comprobar una lectura y escritura cruzadas denegadas, probar la ruta de salud y registrar fecha, entorno, resultado y captura. Si falla un control, se revierte el cambio o se corrige la migración antes de aceptar el despliegue.

El Anexo B detalla el procedimiento y el Anexo F enumera la evidencia disponible.

2.9.1 Medición de rendimiento en producción

El 03/10/2026 se ejecutó scripts/medir_rendimiento_salud.ps1 contra https://xbvjhrfgznljtwxghvka.supabase.co/functions/v1/salud. Se realizaron 30 solicitudes GET secuenciales con curl, pausa de 150 ms entre muestras y límite de 15 s por solicitud. Se registraron marca horaria, código HTTP y tiempo total percibido por el cliente en docs/evidencias/rendimiento_salud_2026-10-03.csv.

Resultado observado: 30/30 respuestas HTTP 200; 29/30 (96,7 %) en 3 s o menos; promedio 0,896 s y percentil 90, 0,908 s. Esta muestra describe solo disponibilidad y latencia de la ruta de salud en un cliente y momento determinados. RNF-01 exige que al menos el 90 % de las operaciones principales responda en 3 s; ese requisito permanece sin aprobar porque faltan mediciones autenticadas de consultas, altas y modificaciones con usuarios de prueba.

CAPÍTULO 3. CONCLUSIONES Y RECOMENDACIONES

3.1 Conclusiones

1. Análisis: se identificaron actores, requisitos funcionales y no funcionales, alcance y flujo de trabajo (2.2 y 2.3). El levantamiento sigue siendo parcial porque la Tabla 2 contiene referencias externas y todavía faltan mediciones propias de EIVET.

2. Diseño: se definieron arquitectura, entidades, relaciones, permisos, contrato API y wireframes (2.4 y figuras 2–5). El diseño es coherente con el repositorio revisado, sujeto a comprobar el esquema y las políticas efectivamente aplicadas en el proyecto remoto.

3. Implementación: el repositorio contiene panel web y aplicación móvil para propietarios, mascotas, citas, consultas, historial, vacunas y tratamientos (2.6). El CRUD y algunas rutas tienen código local, pero no se acreditó todavía su ejecución completa en producción.

4. Seguridad: se integró Supabase Auth, validación de formularios, roles del personal y scripts RLS (2.7). No puede darse por satisfecha la separación entre propietarios hasta ejecutar y documentar pruebas negativas con dos cuentas en producción.

5. Pruebas: la matriz distingue resultados esperados, observados y pendientes (2.8). La suite local de Flutter terminó con siete pruebas aprobadas; la web rechazó credenciales inválidas y crear-propietario devolvió 401 sin token. En salud, GET y HEAD cumplieron, mientras POST evidenció una discrepancia. Ninguno de esos resultados sustituye las pruebas de los cuatro requisitos Must en producción ni verifica RNF-01.

6. Despliegue y documentación: la web pública responde en Netlify y se documentaron pasos de reconstrucción, despliegue y uso (2.9 y anexos). Un

APK final instalado, el despliegue sincronizado de la función de salud y la reproducción del manual por una segunda persona siguen por verificar.

3.2 Recomendaciones

Aplicar y auditar las migraciones RLS pendientes; crear dos cuentas ficticias de propietario y conservar evidencia de que cada una solo puede leer sus registros y no puede modificarlos indebidamente. Repetir los casos con administrador y veterinario para verificar los permisos previstos.

Publicar la versión de la función salud que rechaza POST, ejecutar de nuevo scripts/verify_salud.ps1 y adjuntar salida y captura fechada. Completar las pruebas funcionales del CRUD web y móvil y obtener una APK arm64 instalable.

Medir RNF-01 en producción sobre operaciones principales con sesiones autorizadas, una muestra y condiciones definidas, guardando tiempos individuales y criterio de aceptación. Cronometrar también el proceso manual actual de EIVET para reemplazar los valores no medidos de la Tabla 2.

Solicitar acceso de lectura al tablero privado para el tribunal, confirmar los datos institucionales de portada y ejecutar una revisión del manual con otra persona. Ensayar una demostración de siete minutos con cuentas ficticias, red estable, ruta de salud y evidencias ya preparadas.

BIBLIOGRAFÍA CONSULTADA

AHMED, M. A.; OTSYINA, H. R.; JOHNSON, S. A.; ASARE, D. A.; SHABAN, S.; EMIKPE, B. O. User perception and attitude toward electronic medical record system in veterinary practice in Ghana [en línea]. PAMJ-One Health, 12, artículo 14, 2023. DOI: 10.11604/pamj-oh.2023.12.14.40720. Disponible en: <https://www.one-health.panafrican-med-journal.com/content/article/12/14/full/> [consulta: 03 octubre 2026].

ANHOLT, R. M.; BEREZOWSKI, J.; MACLEAN, K.; RUSSELL, M. L.; JAMAL, I.; STEPHEN, C. The application of medical informatics to the veterinary management programs at companion animal practices in Alberta, Canada: a case study. Preventive Veterinary Medicine [en línea], 113(2), 165–174, 2014. DOI: 10.1016/j.prevetmed.2013.11.005. Disponible en: <https://pubmed.ncbi.nlm.nih.gov/24299904/> [consulta: 03 octubre 2026].

DAYSMART VET. Cloud-Based Veterinary Practice Management Software [en línea]. s. f. Disponible en: <https://www.daysmart.com/vet/> [consulta: 03 octubre 2026].

FIELDING, R. T. Architectural Styles and the Design of Network-based Software Architectures [en línea]. Tesis doctoral. University of California, Irvine, 2000. Disponible en: <https://ics.uci.edu/~fielding/pubs/dissertation/abstract.htm> [consulta: 03 octubre 2026].

FLUTTER. Web support for Flutter [en línea]. s. f. Disponible en: <https://docs.flutter.dev/platform-integration/web> [consulta: 03 octubre 2026].

JONES-DIETTE, J.; ROBINSON, N. J.; COBB, M.; BRENNAN, M. L.; DEAN, R. S. Accuracy of the electronic patient record in a first opinion veterinary practice [en línea]. Preventive Veterinary Medicine, 148, 121–126, 2017. DOI: 10.1016/j.prevetmed.2016.11.014. Disponible en:

<https://doi.org/10.1016/j.prevetmed.2016.11.014> [consulta: 03 octubre 2026].

KANBAN GUIDES. Open Guide to Kanban [en línea]. Versión 2025.7. 2025. Disponible en: <https://kanbanguides.org/open-guide-to-kanban/2025.7/pdf/open-guide-to-kanban.en-us.pdf> [consulta: 03 octubre 2026].

SUPABASE. Auth [en línea]. s. f. Disponible en: <https://supabase.com/docs/guides/auth> [consulta: 03 octubre 2026].

SUPABASE. Data REST API [en línea]. s. f. Disponible en: <https://supabase.com/docs/guides/api> [consulta: 03 octubre 2026].

SUPABASE. Row Level Security [en línea]. s. f. Disponible en: <https://supabase.com/docs/guides/database/postgres/row-level-security> [consulta: 03 octubre 2026].

VETCLOUD. Software de Gestión Integral para Clínicas y Hospitales Veterinarios [en línea]. s. f. Disponible en: <https://vetcloud.cl/> [consulta: 03 octubre 2026].

VETICLOUD. Software de gestión para clínicas veterinarias [en línea]. s. f. Disponible en: <https://veticloud.com/es> [consulta: 03 octubre 2026].

ANEXOS

Anexo A. MANUAL DE USUARIO

A.1 Acceso. Abra https://eivetveterinaria.netlify.app/ desde un navegador actualizado. El personal ingresa con una cuenta habilitada por administración. La aplicación móvil Android se instala desde el APK generado para el dispositivo; los propietarios usan sus credenciales de Supabase Auth. Nunca utilice datos clínicos reales en demostraciones.

A.2 Personal administrador y veterinario. Tras iniciar sesión, use el menú lateral para localizar propietarios, mascotas, citas veterinarias, consultas e historial, vacunas y tratamientos. Seleccione un registro antes de editarlo; compruebe titular, mascota y fecha antes de guardar. El administrador puede crear cuentas de propietario mediante el flujo autorizado. La gestión de mascotas corresponde al administrador y al veterinario.

A.3 Propietario. En la app móvil, abra Mascotas para consultar la ficha propia; use Citas para solicitar o consultar una cita y las pestañas de la tarjeta para ver historial, vacunas y tratamientos disponibles. El propietario no debe registrar, editar ni eliminar mascotas. Si una sección carece de registros, se presenta un estado vacío.

A.4 Errores y ayuda. Si el inicio de sesión falla, revise correo y contraseña o solicite restablecimiento al administrador. Si no aparece una mascota, confirme la cuenta y comunique al personal su identificador; no intente acceder con la cuenta de otra persona. Ante un error de red, reintente cuando haya conexión y registre hora y operación.

Anexo B. MANUAL DE INSTALACIÓN Y DESPLIEGUE

B.1 Requisitos. Instale Git y Flutter 3.44.8 con Dart 3.12.2; para Android agregue Android SDK y un equipo autorizado por USB. Se requiere un proyecto Supabase y acceso de despliegue a Netlify. Las credenciales sensibles se administran fuera del repositorio.

B.2 Preparación local. Ejecute git clone https://github.com/arturov7/Veterinaria-Eivet.git, ingrese al directorio del proyecto y ejecute flutter pub get. Copie .env.example a .env y complete APP_MODE, SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY con los datos de su entorno. No publique .env.

B.3 Base de datos y funciones. Aplique los SQL indicados en docs/SUPABASE_EIVET.md en el orden allí descrito. Verifique tablas, índices, RLS y políticas con cuentas ficticias antes de cargar información. Instale la RPC de supabase/salud.sql; despliegue las Edge Functions del directorio supabase/functions y configure sus secretos de servidor. Vuelva a ejecutar las comprobaciones de salud y de aislamiento de cuentas.

B.4 Ejecución y compilación. Para web local: flutter run -d chrome --dart-define-from-file=.env. Para Android: flutter devices y flutter run -d IDENTIFICADOR_REAL --dart-define-from-file=.env; reemplace IDENTIFICADOR_REAL por la columna ID, sin escribir signos de menor o mayor. Para APK arm64: flutter build apk --release --target-platform android-arm64 --dart-define-from-file=.env. Para web: flutter build web --release --dart-define-from-file=.env. Estos comandos se documentan para reproducir el despliegue; no se ejecutaron en esta revisión.

B.5 Netlify. Conecte la rama main de GitHub; netlify.toml ejecuta scripts/build_netlify.sh y publica build/web. En la configuración del sitio cargue SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY. Tras desplegar, abra

la URL pública, pruebe inicio de sesión con cuentas ficticias y registre identificador de despliegue, fecha, capturas y resultados de pruebas.

B.6 Comprobación por otra persona. Entregue este manual y acceso temporal a un entorno de prueba a una persona distinta del autor. Pídale registrar sistema operativo, versión Flutter, comandos ejecutados, resultado de la compilación y una captura de inicio de sesión. Esta prueba de reproducibilidad aún no se ha realizado.

Anexo C. REPOSITORIO Y TABLERO

Repositorio: https://github.com/arturov7/Veterinaria-Eivet. Rama de publicación: main. El código contiene lib, scripts, supabase, netlify.toml, pubspec.yaml y documentación técnica. El README ofrece una entrada rápida.

Tablero Kanban: https://github.com/users/arturov7/projects/2/views/1?layout=board. La Figura 1 conserva la captura del tablero; su acceso externo es privado y debe compartirse con el evaluador. El límite de trabajo en curso es de dos tarjetas.

Anexo D. URL Y PAQUETE INSTALABLE

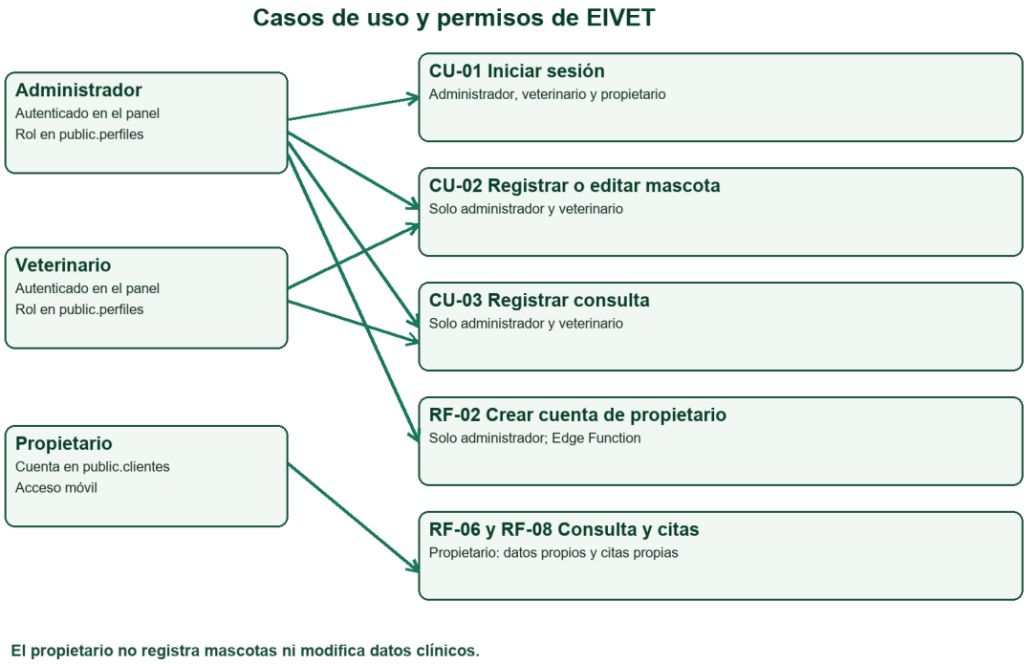
Versión web pública: https://eivetveterinaria.netlify.app/. Ruta de salud: https://xbvjhrfgznljtwxghvka.supabase.co/functions/v1/salud. La apertura pública del panel y 30 respuestas GET de salud se comprobaron el 03/10/2026; el 06/10/2026 se repitió la comprobación de métodos y persistió la discrepancia de POST.

El APK arm64 se obtiene con el comando del Anexo B en build/app/outputs/flutter-apk/app-release.apk. No se adjunta un APK verificado en este borrador; debe compilarse, instalarse y registrar versión, huella y evidencia de uso antes de la entrega.

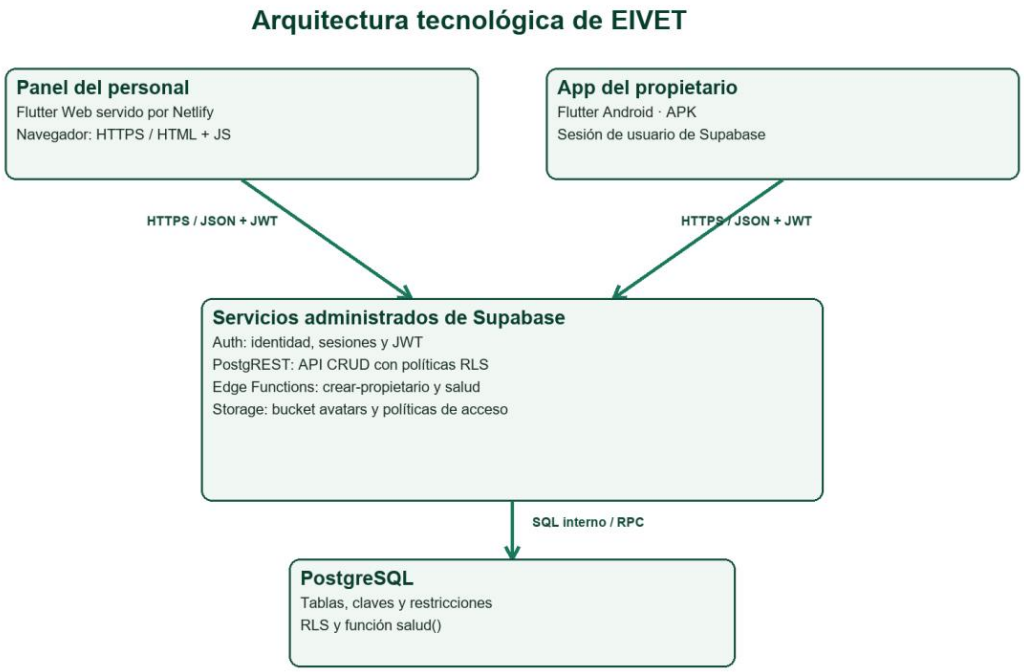
Anexo E. DIAGRAMAS Y TRAZABILIDAD

Las Figuras 2–5 del cuerpo del documento muestran casos de uso, componentes y protocolos, modelo de datos y wireframes. La Tabla 3 relaciona los seis objetivos con evidencias y conclusiones; las Tablas 9–11 detallan arquitectura, diccionario y contrato API. Esos elementos integran el anexo técnico y deben contrastarse con el proyecto Supabase desplegado antes del cierre.

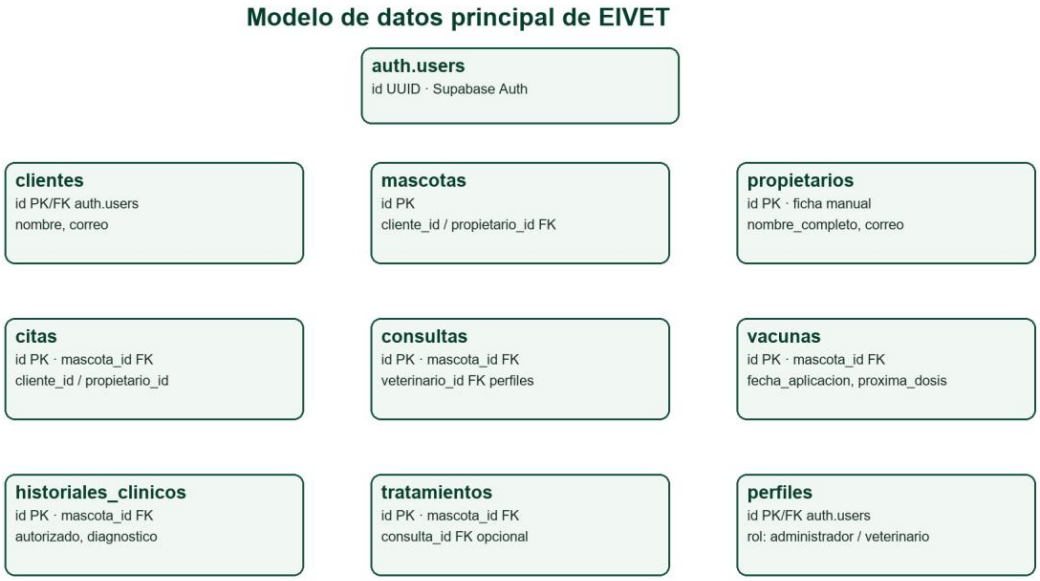
E.1 Casos de uso y permisos.



E.2 Componentes y protocolos.



E.3 Modelo de datos.



RELACIONES DEL ESQUEMA

auth.users.id → clientes.id y perfiles.id

mascotas.cliente_id → clientes.id; mascotas.propietario_id → propietarios.id

citas, consultas, vacunas, historiales y tratamientos → mascotas.id

tratamientos.consulta_id → consultas.id (opcional)

consultas.veterinario_id → perfiles.id (opcional)

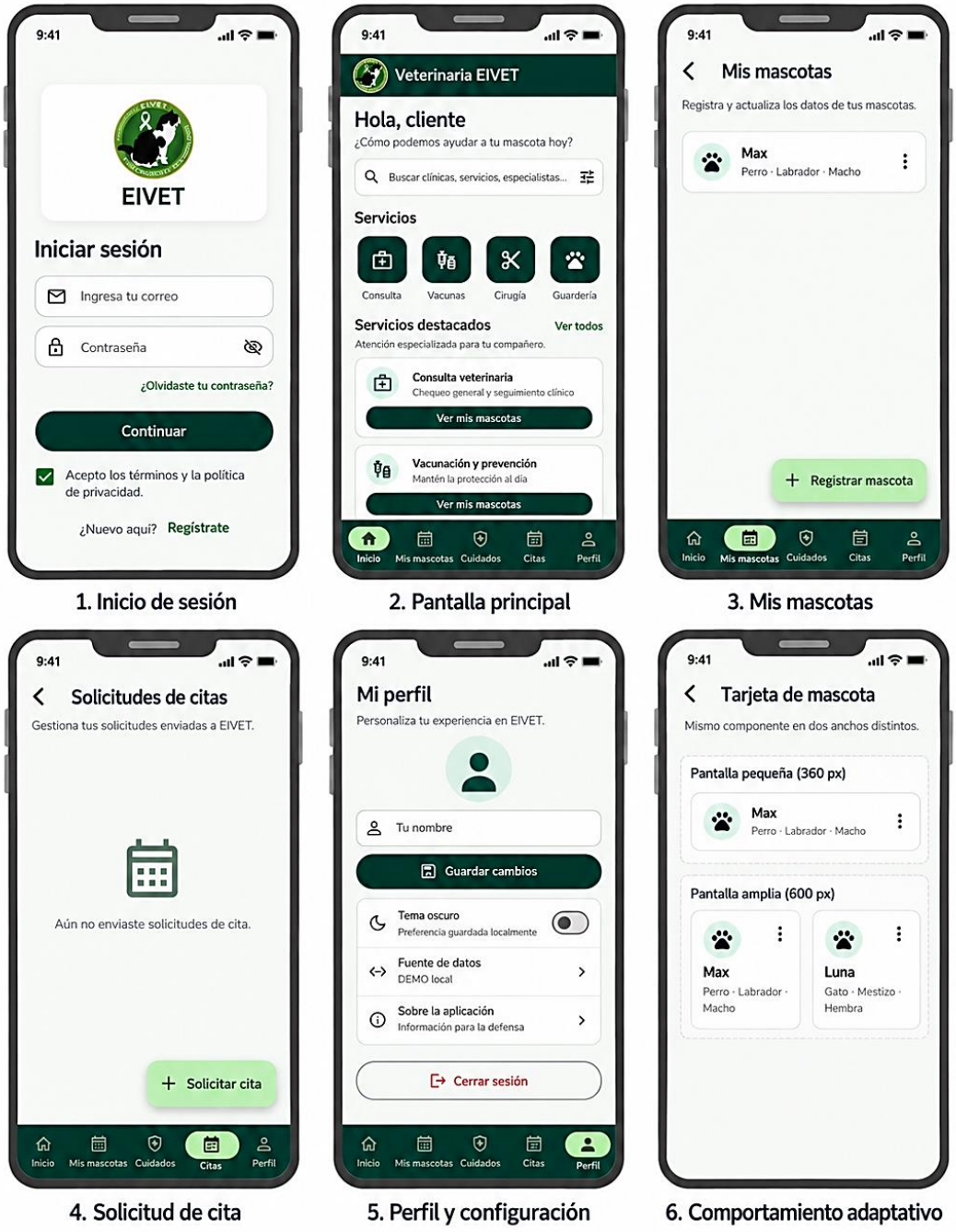
Cada mascota tiene un solo titular según el script 12.

Tablas complementarias: servicios y notificaciones.

E.4 Wireframes iniciales; el botón de registro de mascotas del propietario corresponde a un diseño anterior.

Figura 5. Wireframes de las pantallas principales

Aplicación móvil Veterinaria **EIVET**



Fuente: elaboración propia.

Anexo F. EVIDENCIAS, CASOS DE PRUEBA Y DEFENSA

F.1 Evidencia disponible. La Figura 1 documenta el tablero Kanban. docs/PRUEBAS_Y_EVIDENCIAS.md contiene la matriz; docs/evidencias/flutter_test_2026-10-06.txt conserva la salida de siete pruebas locales aprobadas. docs/evidencias/verify_salud_2026-10-06.txt registra GET, HEAD y POST; docs/evidencias/rf01_login_invalido_2026-10-06.jpg muestra el rechazo del login, y docs/evidencias/crear_propietario_sin_token_2026-10-06.txt registra la respuesta 401. docs/evidencias/rendimiento_salud_2026-10-03.csv conserva las 30 muestras de latencia.

F.2 Resultado observado. La suite local aprobó siete pruebas. En producción se observó el rechazo de credenciales inválidas y la respuesta 401 de crear-propietario sin token; no se verificaron efectos sobre datos. En salud, GET y HEAD cumplieron; POST falló porque devolvió HTTP 200 y el código local espera 405. La prueba con dos propietarios, los cuatro Must completos, las capturas de los flujos clínicos y la instalación del APK permanecen sin evidencia de aprobación.

F.3 Ensayo de defensa. Preparar una demostración de siete minutos: problema y objetivo (1 min), arquitectura y seguridad (1 min), inicio de sesión y flujo principal en producción (3 min), caso de permiso denegado y salud (1 min), resultados y límites (1 min). Reservar cinco minutos para preguntas. Cuentas ficticias, red de respaldo, capturas y CSV deben estar disponibles antes del ensayo. El ensayo con una segunda persona todavía no se ha efectuado.